

Especialización en Sistemas Embebidos

Circuitos Lógicos Programables

Trabajo práctico final

Modulador PPM controlado por UART implementado en FPGA

Alumno: Mauro Ariel Gianchino

Fecha: 13/10/2026

Resumen

Se presenta la implementación en una FPGA de un modulador por posición de pulso (PPM) descripto en VHDL. El sistema recibe bytes por un puerto UART y, por cada uno, genera un único pulso cuya posición dentro de una trama periódica de 2,56 ms codifica su valor (0 a 255), junto con una señal de sincronismo que marca el comienzo de cada trama. El diseño se simuló con ModelSim, se sintetizó e implementó con Vivado para la placa Arty Z7-10 usando menos del 0,4 % de la lógica disponible y se verificó en hardware con un analizador lógico.

Índice

1. Introducción y objetivo **2**

2. Modulación por posición de pulso **2**

3. Descripción del sistema **2**
3.1. Parámetros de diseño . . . 2
3.2. Interfaz . . . 3
3.3. Forma de onda de salida . . . 3

4. Arquitectura **4**
4.1. Diagrama en bloques . . . 4
4.2. Archivos y bloques . . . 5
4.3. El módulo `mod_ppm` . . . 5

5. Simulaciones **6**
5.1. Entorno . . . 6
5.2. Testbenches . . . 6
5.2.1. Testbench 1: `mod_ppm_tb1_basico` . . . 7
5.2.2. Testbench 2: `mod_ppm_tb2_secuencia` . . . 8
5.2.3. Testbench 3: `mod_ppm_tb3_primero` . . . 9
5.2.4. Testbench 4: `mod_ppm_tb4_rdylargo` . . . 10
5.2.5. Testbench 5: `uart_top_tb` . . . 11

6. Síntesis e implementación **11**
6.1. Archivos . . . 11
6.2. Temporización . . . 12
6.3. Uso de recursos . . . 12

7. Verificación en hardware **13**
7.1. Montaje . . . 13
7.2. Mediciones . . . 13
7.2.1. Período entre pulsos de sincronización . . . 13
7.2.2. Ancho de pulso de sincronización . . . 14
7.2.3. Ancho de pulso de dato . . . 14
7.2.4. Envío de datos . . . 15
7.2.5. Resumen de mediciones . . . 15
7.3. Prueba del reset . . . 16

8. Conclusiones **16**

1. Introducción y objetivo

Este trabajo implementa en una FPGA un modulador PPM (*Pulse Position Modulation*) descripto en VHDL. El sistema recibe bytes por un puerto UART y, por cada byte recibido, genera un único pulso cuya posición dentro de una trama periódica codifica el valor del byte. Una segunda salida marca el comienzo de cada trama y sirve de referencia de tiempo para medir la posición del pulso.

El diseño se simuló con ModelSim, se sintetizó e implementó con Vivado 2018.3 para la placa Digilent Arty Z7-10, y se verificó en hardware con un analizador lógico, enviando caracteres desde una PC a través de un adaptador USB-serie.

2. Modulación por posición de pulso

En la modulación PPM la información se codifica en el instante en que aparece un pulso dentro de una ventana de tiempo, llamada trama, y no en su amplitud ni en su ancho: el pulso tiene siempre la misma amplitud y el mismo ancho. La trama se divide en M ranuras de igual duración (ancho) τ y el dato d elige la ranura en la que se coloca el pulso. La duración de la trama es

$$T = M\,\tau. \tag{1}$$

3. Descripción del sistema

3.1. Parámetros de diseño

Tabla 1. Parámetros del diseño.

Parámetro	**Valor**
Placa / FPGA	Digilent Arty Z7-10
Frecuencia de clock	125 MHz; período de 8 ns
Duración de la ranura τ (= ancho del pulso)	10 µs = 1250 ciclos de clock
Cantidad de ranuras M	256: una por cada valor de un byte (0 a 255)
Trama T	$256 \times 10\,\mu s = 2{,}56\,ms = 320\,000$ ciclos (390 tramas por segundo)
Pulso de sincronismo	1 µs = 125 ciclos, al comienzo de cada trama
UART	115 200 baudios, 8N1
Posición del pulso para el dato d	$d \times \tau$

Se eligió 10 µs como resolución temporal ya que equivale a 240 muestras de un analizador lógico de 24 MHz, lo que permite medir la posición con facilidad. Los criterios de diseño adoptados son:

- **Un pulso por byte.** Si al comenzar una trama no hay un byte nuevo esperando, la trama va vacía (sin pulso).

- **Carga al inicio de trama.** El byte recibido durante una trama se guarda y recién se usa en la trama siguiente, de modo que ninguna trama tenga un pulso a medias ni dos pulsos.
- **El primero gana.** Si llegan varios bytes durante una trama, se modula el primero y se descartan los demás.
- **Sin ranura de seguridad.** El byte 255 usa la última ranura. Si en tramas consecutivas se usaran el 255 y el 0, los dos pulsos quedarían pegados y se verían como uno solo.

3.2. Interfaz

El sistema se presenta como un único módulo, `uart_top`, con cinco puertos (figura 1). Recibe tres señales externas: el clock de 125 MHz de la placa, un reset manual y la línea serie del UART por la que llegan los bytes a modular. Entrega dos salidas: `ppm_pin`, con el pulso cuya posición codifica el byte recibido, y `sync_pin`, que marca el comienzo de cada trama y sirve de referencia para medir esa posición. En la tabla 2 se indica la función de cada puerto y el pin de la FPGA al que está asignado.

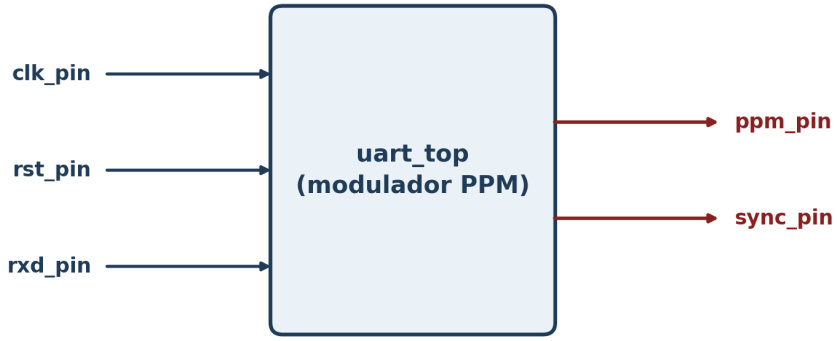


Figura 1. Módulo `uart_top`.

Tabla 2. Puertos del top y asignación de pines.

Puerto	I/O	Pin	Descripción
`clk_pin`	Entrada	H16	Clock de 125 MHz
`rst_pin`	Entrada	L19 (BTN3)	Reset activo en alto
`rxd_pin`	Entrada	Y18 (PMOD JA, pin 1)	Entrada UART
`ppm_pin`	Salida	Y16 (PMOD JA, pin 3)	Salida PPM
`sync_pin`	Salida	Y17 (PMOD JA, pin 4)	Sincronización

3.3. Forma de onda de salida

Para un byte d, `ppm_pin` vale 1 durante la ranura d de la trama, es decir, desde $d\,\tau$ hasta $(d+1)\,\tau$ medidos desde el flanco de subida de `sync_pin`:

$$t_{\text{ret}} = d \cdot \tau = d \times 10\,\mu\text{s}. \tag{2}$$

Por ejemplo, el carácter `'A'` (ASCII 65) produce un pulso de 10 µs que comienza 650 µs después del sync. La posición se mide siempre entre los flancos de subida de ambas señales.

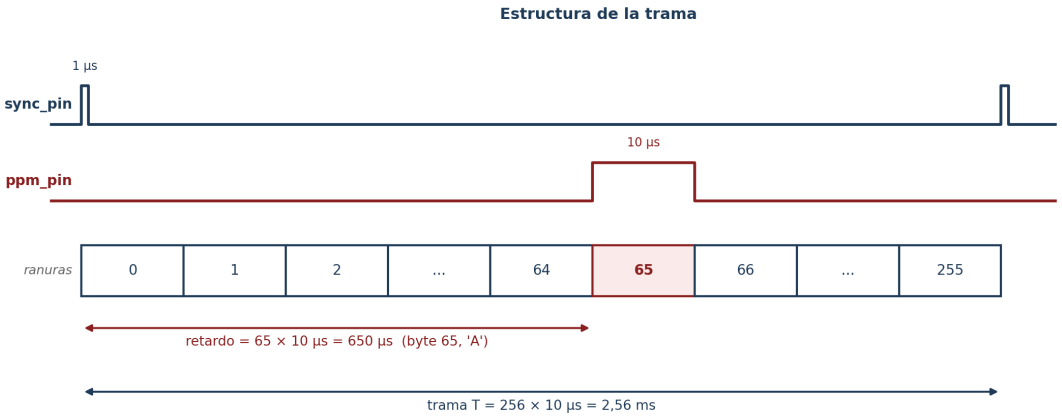


Figura 2. Estructura de la trama. `sync_pin` marca el comienzo; `ppm_pin` vale 1 durante la ranura cuyo número es el valor del byte.

4. Arquitectura

4.1. Diagrama en bloques

El diseño se organiza en tres niveles: `uart_top` conecta los pines de la placa y fija los parámetros del UART; `uart_ppm` une el receptor serie con el modulador y sincroniza el reset; `mod_ppm` convierte el byte recibido en el pulso PPM.

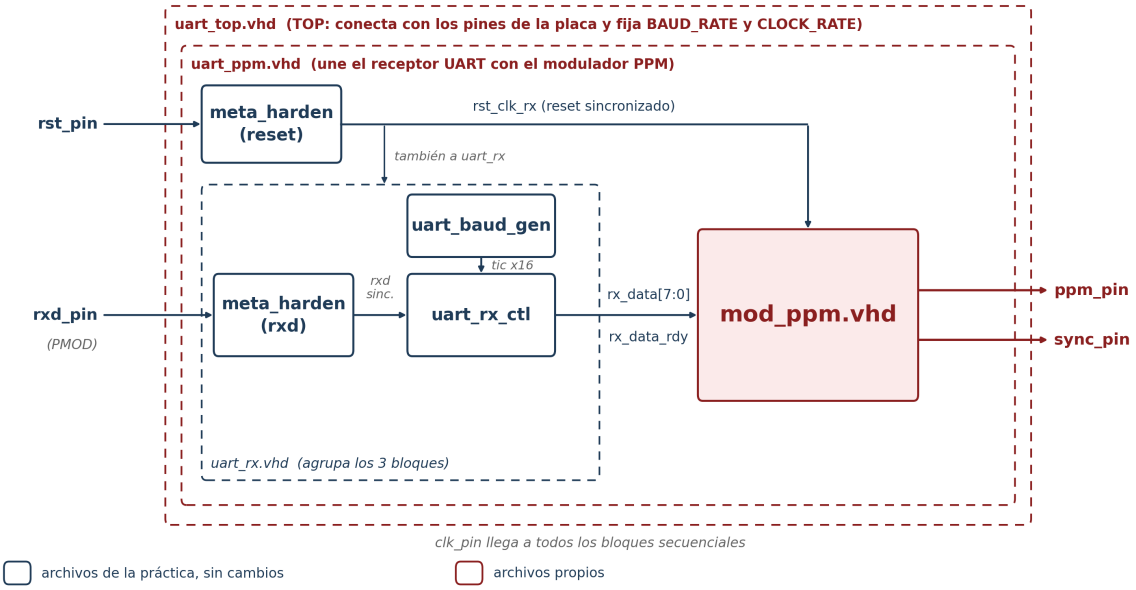


Figura 3. Diagrama en bloques y archivos que contiene cada nivel. En azul, los archivos de la Práctica 2 sin cambios; en rojo, los propios.

4.2. Archivos y bloques

Los bloques del receptor UART y el sincronizador provienen de la Práctica 2 de la materia y se usan sin modificaciones. `uart_top` y `uart_ppm` siguen la estructura de `uart_top` y `uart_led` de esa práctica; `mod_ppm` es de elaboración propia y reemplaza al bloque que mostraba el byte en los LEDs.

Tabla 3. Archivos VHDL del diseño.

Archivo	Función	Origen
`uart_top.vhd`	Puertos externos y parámetros `BAUD_RATE` y `CLOCK_RATE`	Propio
`uart_ppm.vhd`	Une el UART con el modulador; sincroniza el reset	Propio
`mod_ppm.vhd`	Modulador PPM	Propio
`uart_rx.vhd`	Receptor UART: agrupa sincronizador, generador de tics y control	Práctica 2
`uart_rx_ctl.vhd`	Máquina de estados de la recepción serie	Práctica 2
`uart_baud_gen.vhd`	Generador de tics a 16 veces el baud rate	Práctica 2
`meta_harden.vhd`	Sincronizador de dos flip-flops para entradas asíncronas	Práctica 2

4.3. El módulo `mod_ppm`

`mod_ppm` tiene tres procesos sincrónicos, todos con reset sincrónico activo en alto:

- **Contadores.** Dos contadores en cascada: uno cuenta los ciclos dentro de la ranura (de 0 a 1249) y otro las ranuras dentro de la trama (de 0 a 255). El segundo avanza cuando el primero completa su vuelta; cuando ambos llegan a su máximo comienza una trama nueva.
- **Byte.** Captura el byte en el flanco de subida de `rx_data_rdy_i` y levanta el flag `byte_waiting`. Al comienzo de cada trama, si hay un byte esperando lo pasa al registro de comparación, baja el flag y marca la trama como con mensaje; si no hay, la marca como vacía. Para esto usa el flag `empty_msg`.
- **Salidas.** `ppm_o` vale 1 cuando la trama tiene mensaje y la ranura actual es igual al byte; `sync_o` vale 1 durante los primeros 125 ciclos de la trama.

Tabla 4. Parámetros y señales internas de `mod_ppm`.

Señal / generic	Función
`CYCLES_PER_SLOT` = 1250	Ciclos de clock por ranura (10 µs a 125 MHz)
`NUM_SLOTS` = 256	Ranuras por trama
`W_SYNC` = 125	Ciclos que dura `sync_o` en alto (1 µs)
`cycle_count`, `window_count`	Contadores de ciclos y de ranuras
`old_rx_data_rdy`	Valor anterior de `rdy`, para detectar su flanco de subida
`arrival_byte`	Último byte capturado
`byte_waiting`	Hay un byte capturado que todavía no se usó
`compare_byte`	Byte que se compara con el contador de ranuras en la trama actual
`empty_msg`	1: la trama va vacía; 0: la trama lleva un pulso

5. Simulaciones

5.1. Entorno

Las simulaciones se realizaron con ModelSim Intel FPGA Edition 2020.1. Los testbenches de `mod_ppm` usan *generics* reducidos (4 ciclos por ranura y pocas ranuras) para que una trama completa se vea en pocas decenas de ciclos, con el mismo comportamiento que con los valores reales. El testbench del sistema completo usa los valores reales de 125 MHz y 115 200 baudios.

5.2. Testbenches

Se desarrollaron cinco testbenches (tabla 5). Los cuatro primeros prueban `mod_ppm` de forma aislada, inyectando directamente el byte y la señal `rdy` que entregaría el receptor UART, y cada uno se enfoca en un comportamiento del módulo: la posición del pulso para un byte, el desplazamiento del pulso entre tramas consecutivas, la regla de que gana el primer byte cuando llegan varios en la misma trama, y la captura de un único byte aunque `rdy` permanezca en alto durante varias tramas. El último testbench prueba el sistema completo: envía tres bytes por la línea serie a 115 200 baudios y comprueba toda la cadena, desde el receptor UART hasta el pulso de salida. En todos los casos el resultado fue el esperado.

Tabla 5. Testbenches y resultados.

Testbench	**Qué verifica**	**Resultado**
`mod_ppm_tb1_basico`	Un byte (de valor 10) llega durante la primera trama; trama de 12 ranuras.	Pulso en la ranura 10 de la trama siguiente.
`mod_ppm_tb2_secuencia`	Bytes 0, 1, 2 y 3, uno por trama (4 ranuras).	Un pulso por trama, en las ranuras 0, 1, 2 y 3.
`mod_ppm_tb3_primero`	Dos bytes (5 y 2) en la misma trama.	Un solo pulso, en la ranura 5; el 2 se descarta.
`mod_ppm_tb4_rdylargo`	`rdy` en alto durante varias tramas con el byte 3.	Un solo pulso; las tramas siguientes van vacías.
`uart_top_tb`	Sistema completo: los bytes 5, 65 y 122 se envían por la línea serie, uno por trama.	Un pulso por byte, en la trama siguiente a su llegada, en las ranuras 5, 65 y 122.

5.2.1. Testbench 1: `mod_ppm_tb1_basico`

Este testbench envía un único byte de valor 10 a un `mod_ppm` con 12 ranuras de 4 ciclos, o sea tramas de 4 800 ns. En la figura 4 se ve que el pulso de `rdy` que entrega el byte cae durante la primera trama, que va vacía, y que el pulso de `ppm` aparece recién en la trama siguiente.

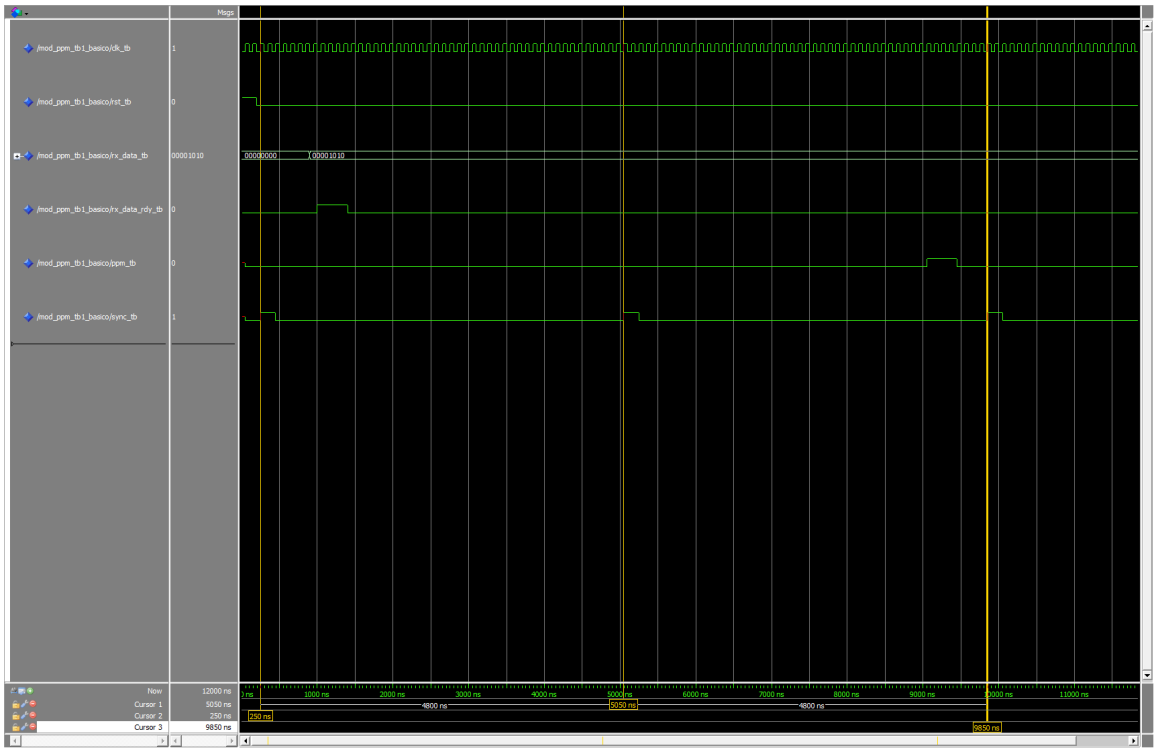


Figura 4. Simulación de `mod_ppm_tb1_basico`: el byte 10 llega durante la primera trama y el pulso aparece en la ranura 10 de la trama siguiente.

5.2.2. Testbench 2: `mod_ppm_tb2_secuencia`

Se envían los bytes 0, 1, 2 y 3, uno por trama, a un módulo de 4 ranuras de 4 ciclos (tramas de 1 600 ns, que los cursores de la figura 5 confirman entre un `sync` y el siguiente). Cada byte, acompañado por su pulso de `rdy`, se usa en la trama que sigue, y el pulso de `ppm` se corre una ranura hacia la derecha en cada trama: sale en la ranura 0, 1, 2 y 3. El primero coincide con el flanco del `sync`, porque el byte 0 ocupa la primera ranura. Esta es la simulación que mejor muestra la modulación, con el pulso moviéndose según el dato.

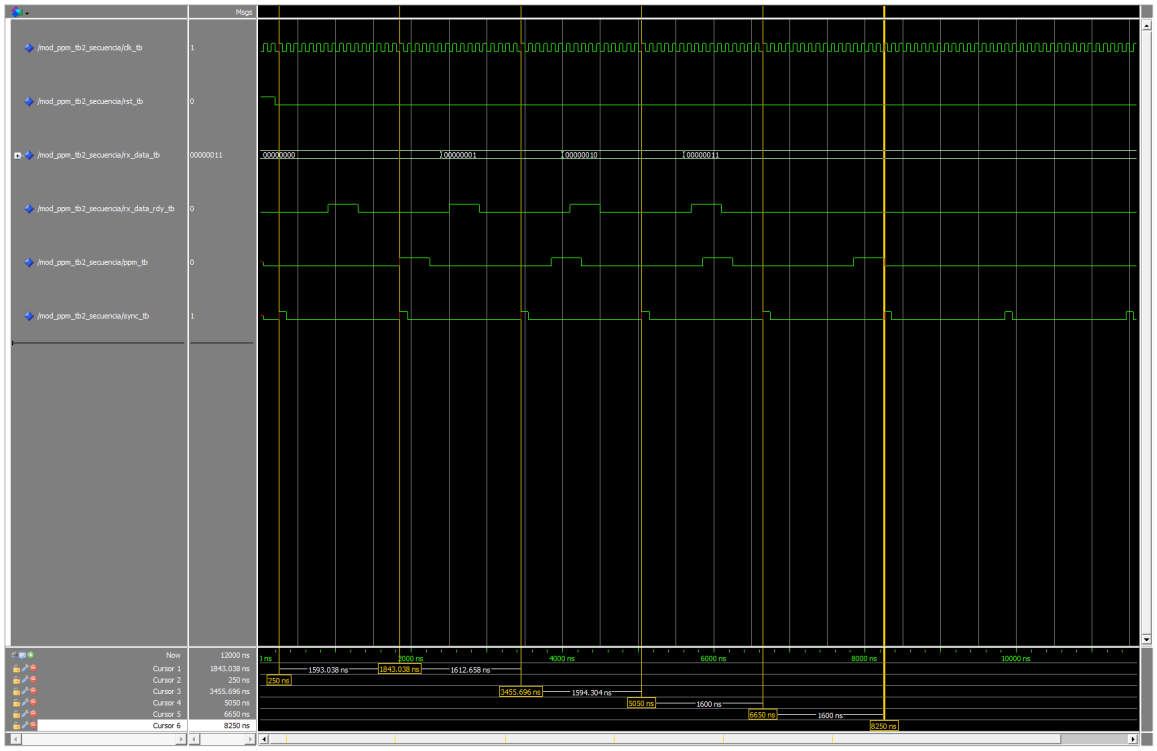


Figura 5. Simulación de `mod_ppm_tb2_secuencia`: con los bytes 0, 1, 2 y 3, el pulso de `ppm` se desplaza una ranura en cada trama.

5.2.3. Testbench 3: `mod_ppm_tb3_primero`

Este testbench verifica la regla de que gana el primer byte. Durante la primera trama llegan dos bytes seguidos, el 5 (`00000101`) y el 2 (`00000010`), cada uno con su pulso de `rdy` (figura 6). En la trama siguiente, que empieza en 3 450 ns, aparece un único pulso de `ppm`, en la ranura 5. No hay ningún pulso en la ranura 2 ni en las tramas posteriores, de modo que el segundo byte se descartó, y cada trama dura 3 200 ns, como indican los cursores sobre los `sync`.

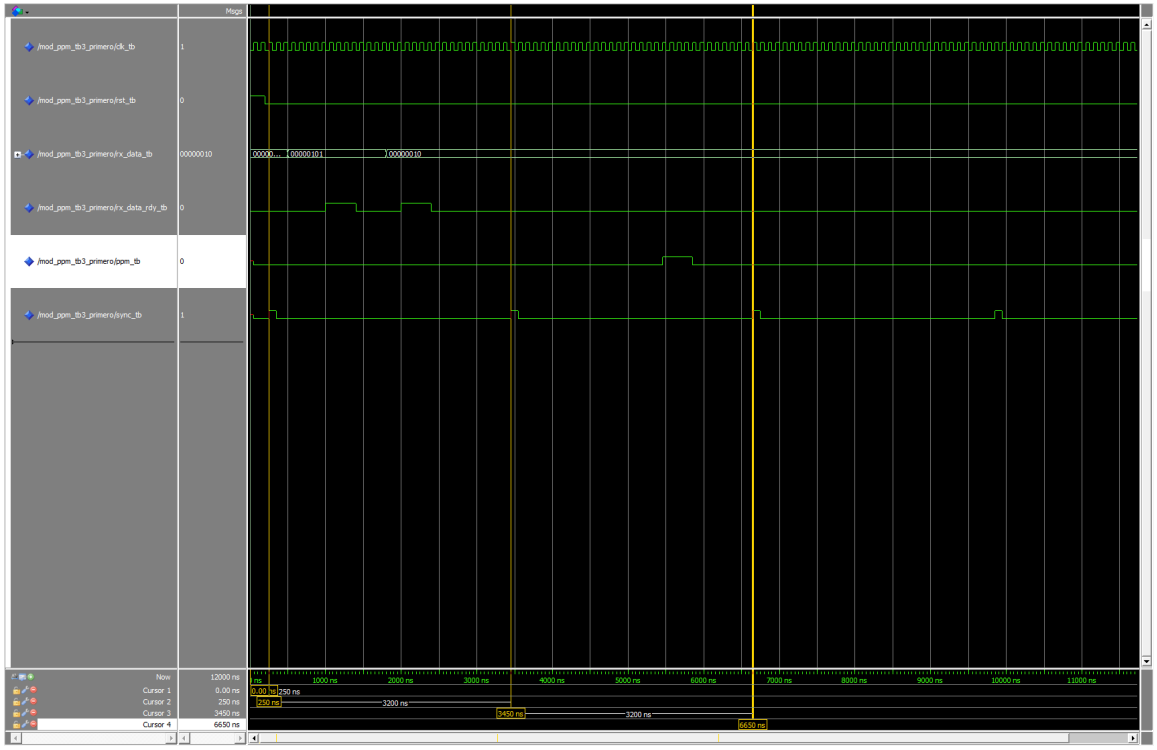


Figura 6. Simulación de `mod_ppm_tb3_primero`: con los bytes 5 y 2 en la misma trama, solo se modula el primero.

5.2.4. Testbench 4: `mod_ppm_tb4_rdylargo`

En este caso el byte 3 llega con `rdy` en alto durante 8,7 µs, que es la duración de un bit del UART a 115 200 baudios, y en este testbench abarca tres tramas (figura 7). A pesar de eso, `ppm` produce un único pulso, en la ranura 3 de la segunda trama, y la tercera trama va vacía aunque `rdy` siga en alto durante buena parte de ella. Esto muestra que el byte se captura solo en el flanco de subida de `rdy`, y no mientras la señal permanece activa, lo que evita repetir el mismo pulso en las tramas siguientes.

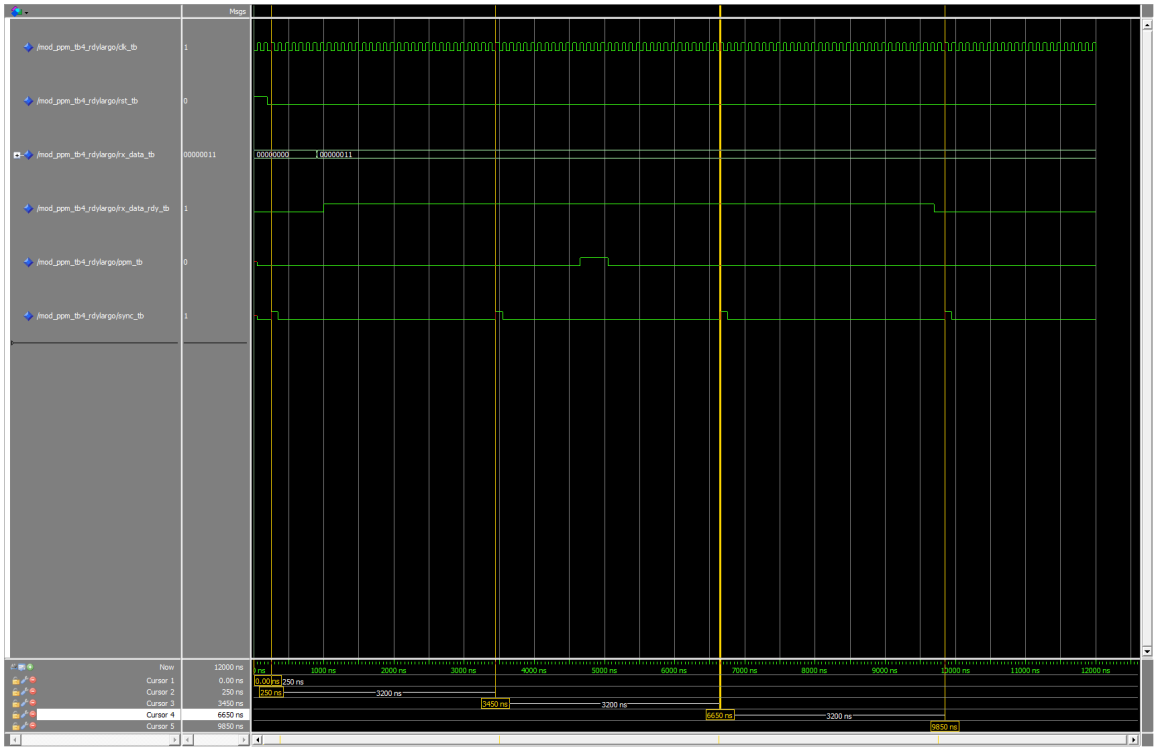


Figura 7. Simulación de `mod_ppm_tb4_rdylargo`: con `rdy` en alto durante varias tramas, el byte genera un solo pulso.

5.2.5. Testbench 5: `uart_top_tb`

Este testbench prueba el sistema completo con los valores reales: un clock de 125 MHz y una línea serie de 115 200 baudios, por la que se envían tres bytes sucesivos, cada uno con su bit de start, los 8 bits de datos (primero el menos significativo) y el bit de stop. Los bytes son el 5, el 65 (`'A'`) y el 122 (`'z'`), enviados a partir de los 100 µs, 3 000 µs y 6 000 µs, de modo que cada uno llega durante una trama distinta, cuando el anterior ya se está modulando. Como los tres valores están bien separados, en la figura 8 se ve cómo el pulso se desplaza hacia la derecha de una trama a la siguiente. Para una mejor apreciación, es recomendable ejecutar la simulación y ampliar (zoom) las zonas de interés.

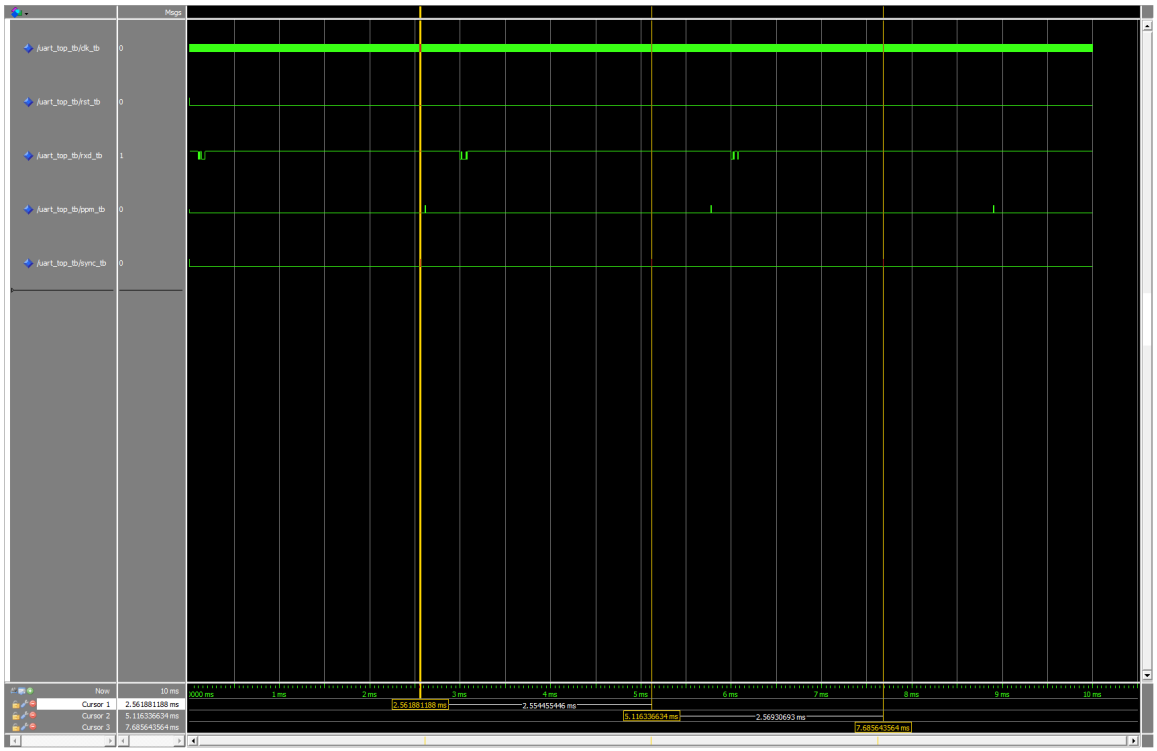


Figura 8. Simulación del sistema completo (`uart_top_tb`) durante 10 ms: los tres bytes enviados por `rxd` producen tres pulsos de `ppm`, cada uno en la trama siguiente y más desplazado respecto del `sync` que la abre (marcado por los cursores, separados unos 2,56 ms). El clock se ve como una banda continua por la escala de tiempo.

6. Síntesis e implementación

6.1. Archivos

Se creó un proyecto de Vivado 2018.3 para la Arty Z7-10 con las siete fuentes VHDL y dos archivos de restricciones:

- `uart_ppm_pins_ArtyZ7.xdc` asigna cada puerto del top a su pin.
- `uart_ppm_timing_ArtyZ7.xdc` contiene una sola restricción, la definición del clock de 125 MHz.

6.2. Temporización

A 125 MHz el diseño cumple la temporización (tabla 6 y figura 9).

Tabla 6. Resumen de temporización después de la implementación (125 MHz).

Parámetro	Resultado
Worst Negative Slack (setup)	3,908 ns
Worst Hold Slack	0,120 ns
Worst Pulse Width Slack	3,500 ns
Endpoints analizados / que fallan	168 / 0

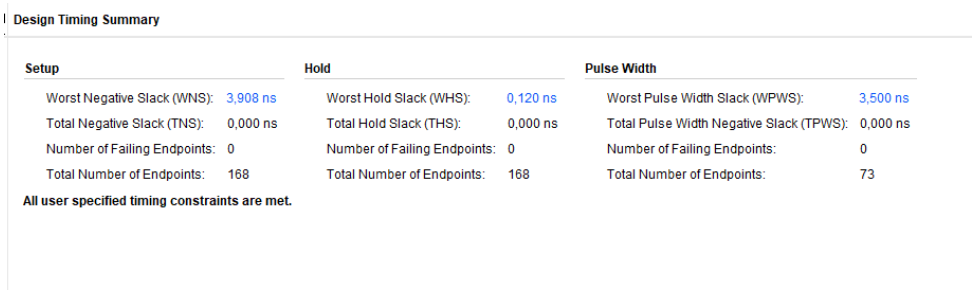


Figura 9. *Design Timing Summary* del diseño implementado.

6.3. Uso de recursos

Las tablas 7 y 8 y la figura 10 muestran el uso de recursos de la FPGA, en total y por módulo.

Tabla 7. Uso de recursos de la FPGA.

Recurso	Utilizado	Disponible	Utilización
LUT	66	17 600	0,38 %
Flip-flops	72	35 200	0,20 %
Entradas/salidas	5	100	5,00 %
BRAM / DSP	0	–	0 %

Tabla 8. Uso de recursos por módulo.

Módulo	LUTs	Flip-flops
`uart_top` (total)	66	72
`mod_ppm`	30	40
`uart_rx`	36	30
`uart_rx_ctl`	28	20
`uart_baud_gen`	8	8
`meta_harden` (rxd)	0	2
`meta_harden` (reset)	0	2

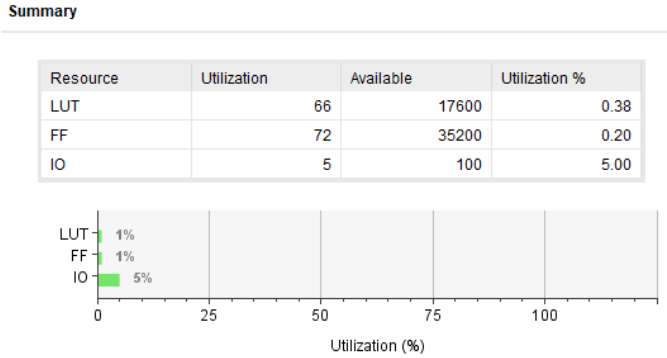


Figura 10. Resumen de utilización de Vivado.

7. Verificación en hardware

Se generó el bitstream y se programó la placa con Vivado. Luego, se realizó el montaje de los elementos para observar la salida de la FPGA.

7.1. Montaje

En la tabla 9 se especifica la conexión a la FPGA del analizador lógico y del adaptador USB-serie, y en la figura 11 se muestra el montaje.

Tabla 9. Conexiones al conector PMOD JA.

Desde	**Hacia (PMOD JA)**
Adaptador USB-serie, TxD	pin 1 (`rxd_pin`)
Adaptador USB-serie, GND	pin 5 (GND)
Analizador lógico, canal PPM_OUTPUT	pin 3 (`ppm_pin`)
Analizador lógico, canal SYNC_OUTPUT	pin 4 (`sync_pin`)
Analizador lógico, GND	pin 11 (GND)

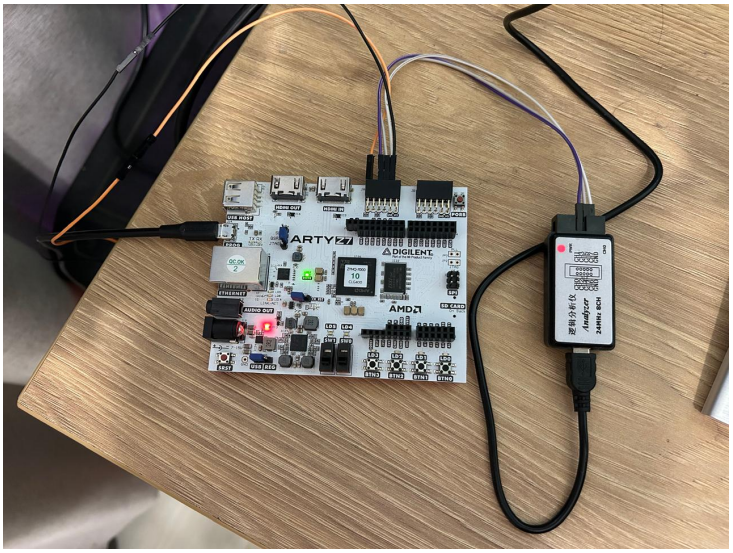


Figura 11. Montaje de prueba: placa Arty Z7-10 configurada, con el adaptador USB-serie y el analizador lógico conectados al conector PMOD JA.

Los caracteres se enviaron desde un terminal serie (PuTTY) por el adaptador USB-serie a 115 200 baudios, 8N1. El analizador lógico muestreó a 24 MHz.

7.2. Mediciones

En esta sección se detallan las mediciones efectuadas con el analizador lógico y las capturas.

7.2.1. Período entre pulsos de sincronización

Se capturó `sync_pin` y se midió el tiempo entre dos flancos de subida consecutivos (figura 12). El resultado fue de 2,559 ms a 2,560 ms, que coincide prácticamente con los 2,56 ms esperados y confirma el funcionamiento de los contadores y del clock.

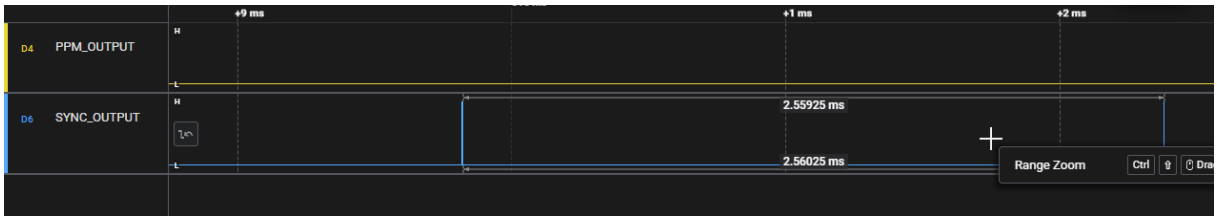


Figura 12. Período entre pulsos de `sync_pin`: 2,56 ms.

7.2.2. Ancho de pulso de sincronización

Con un zoom sobre un solo pulso de `sync_pin` se midió su ancho (figura 13), que resultó de 1 µs. Es el valor esperado, ya que equivale a los 125 ciclos de `W_SYNC` a 125 MHz.

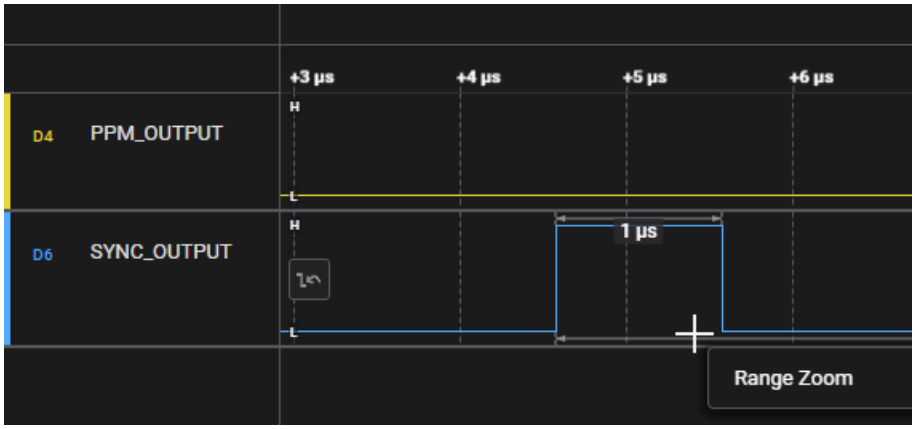


Figura 13. Ancho del pulso de `sync_pin`: 1 µs.

7.2.3. Ancho de pulso de dato

Del mismo modo se midió el ancho de un pulso de `ppm_pin` (figura 14): 10 µs, exactamente una ranura (1250 ciclos), como corresponde al diseño.

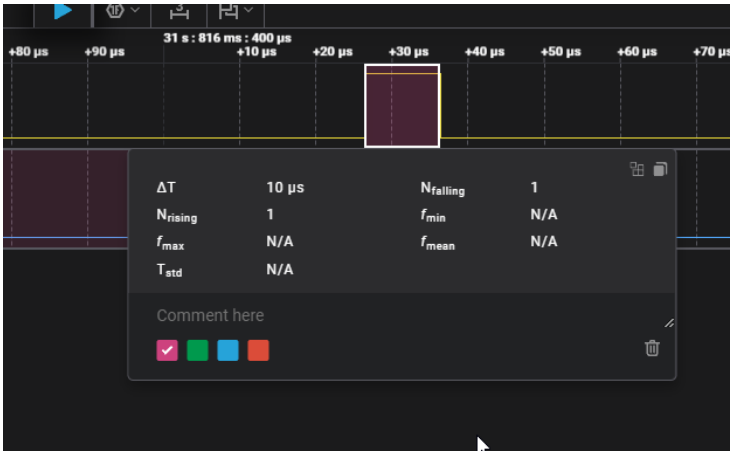


Figura 14. Ancho del pulso de `ppm_pin`: 10 µs.

7.2.4. Envío de datos

Se enviaron caracteres desde el terminal serie y se midió, con dos marcadores, el tiempo entre el flanco de subida de `sync_pin` y el de `ppm_pin`. Para la `'A'` (65) el retardo fue de 649,785 µs (figura 15), frente a los 650 µs esperados. Con otros tres caracteres (figura 16) se obtuvieron 320,011 µs para el byte 32, 480,056 µs para el `'0'` y 490,062 µs para el `'1'`. En todos los casos el retardo coincide con $d \times 10$ µs, y la diferencia entre el `'0'` y el `'1'` es de una ranura (10 µs), lo que verifica la resolución del sistema.

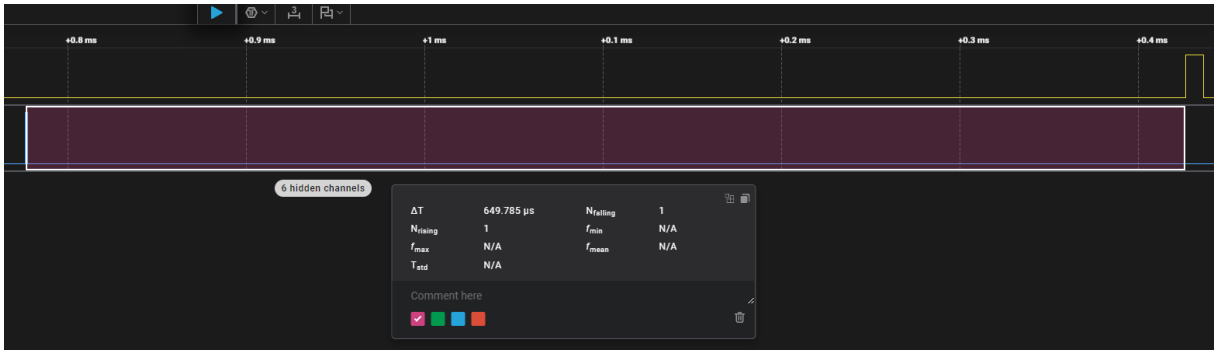


Figura 15. Carácter `'A'` (ASCII 65): el pulso de `ppm_pin` aparece 649,785 µs después del sync.



Figura 16. Retardo entre `sync_pin` y `ppm_pin` para tres caracteres: byte 32 (320 µs), `'0'` (480 µs) y `'1'` (490 µs).

7.2.5. Resumen de mediciones

La tabla 10 reúne las mediciones y las compara con los valores esperados.

Tabla 10. Mediciones con el analizador lógico.

Medición	Esperado	Medido
Período de `sync_pin`	2,56 ms	2,559 ms a 2,560 ms
Ancho del pulso de `sync_pin`	1 µs	1 µs
Ancho del pulso de `ppm_pin`	10 µs	10 µs
Retardo sync-ppm, byte 32 (barra espaciadora)	320 µs	320,011 µs
Retardo sync-ppm, `'0'` (48)	480 µs	480,056 µs
Retardo sync-ppm, `'1'` (49)	490 µs	490,062 µs
Retardo sync-ppm, `'A'` (65)	650 µs	649,785 µs

Las diferencias son menores que 0,25 µs, es decir, menos del 2,5 % de una ranura.

7.3. Prueba del reset

Con el sistema funcionando se mantuvo presionado el botón de reset (BTN3) durante distintos intervalos. Mientras el botón está presionado, `sync_pin` queda en bajo y no se genera ningún pulso en `ppm_pin`; al soltarlo, `sync_pin` retoma de inmediato con una trama nueva. Las ráfagas de pulsos de `ppm_pin` coinciden con los caracteres tecleados y ninguna cae dentro de un reset.

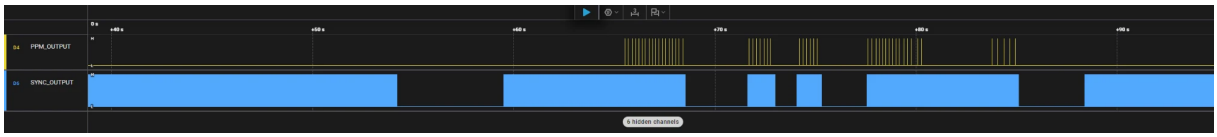


Figura 17. Prueba del reset (escala de segundos): los huecos de `sync_pin` son los intervalos con BTN3 apretado.

8. Conclusiones

Se implementó en la Arty Z7-10 un modulador PPM que convierte cada byte recibido por UART en la posición de un pulso dentro de una trama de 2,56 ms, con una salida de sincronismo para medirla. El diseño ocupa menos del 0,4 % de la lógica de la FPGA, cumple la temporización a 125 MHz con 3,9 ns de margen, y las mediciones en hardware coinciden con los valores teóricos (retardo $= d \times 10$ µs) con diferencias menores que 0,25 µs.